On the Cost of Communication in Divergent Program Evolution

Matched-compute evidence from ARC-AGI-1 that a single agent with a central abstraction library strictly dominates divergent lineages communicating over a constrained channel

Aidan Jones

September 2026

Source: clebran-arc-agi. Evidence reproduction in this directory's REPRODUCE.md; per-experiment data under experiments/2026-07-2* and experiments/2026-08-31_*, experiments/2026-09-22_*, experiments/2026-09-24_* (discovery-bottleneck menu: 0-for-4 at 1M x 5 seeds).

Abstract

We ask whether bandwidth-limited communication between divergent agents — the canonical setting of emergent communication — improves problem-solving in evolutionary program synthesis at *matched compute*. The test is on a small, fully-instrumented substrate: typed programs evolved under pure reward on a 30-task frontier subset of ARC-AGI-1, with candidate-evaluation budgets audited to 0.10% spread across 20 seeds. The baselines are strong enough that a null is trustworthy — a single population with the same total compute, an isolated-lineage ensemble, unconstrained and constrained channels over four payload types, and, critically, a single agent that grows a central abstraction library (a Stitch/DreamCoder-style control). On this substrate the central library sits at the top of the ranking, and the only statistically significant comparison in the study is that it beats the strongest communicating arm by +0.85 evolution-phase solves (~2.2 standard errors). The multi-agent-with-channel hypothesis is falsified by its own strongest control. Lineages *do* diverge (pairwise primitive Jaccard 0.57), but divergence is a *liability* for vocabulary quality: the single agent's mined library is more compositional (topographic similarity 0.33 vs 0.14) and more cross-task reused. The constraint regime is null across all four payloads, because an adaptive learner self-regulates a failing channel regardless of bandwidth.

We then test whether any **mechanism operating inside** the central learner itself — failure-mode dreaming, heterogeneous fitness, task-subset specialization, novelty search — can expand the central library's frontier on the same 30-task subset at matched compute. Four concrete 1M pilots return null (§3.6): the strongest failure-mode-dreaming variant regresses (−1.6, $p = 0.007$), the heterogeneous-fitness variant regresses (−0.80), the task-subset-specialization arms get 0 test-exact discoveries on the hard subset (which B5-4M, at 2× per-task compute, also fails to crack), and the novelty-search arm finds exactly the 6 easy tasks B5-1M also finds (100% overlap, 0 unique). The cheapest reading is that **no mechanism operating inside the current DSL/interpreter/fitness on this frontier will expand B5's reach at 1M, regardless of which axis you vary**; the remaining bets — learned L1 as proposer (#5), operator self-modification (#6), the L3 evoker (#7) — all attack the problem by adding a *new dimension* the lineage can search, rather than redistributing budget across the same DSL. We argue the asset of the work is the matched-compute measurement rig, and that the useful component of the design is the library mechanism, not the multi-agent framing.

1. Introduction

A recurring intuition across emergent communication, cultural evolution of language, and multi-agent learning is that *many divergent learners sharing through a narrow channel* out-

perform monolithic learning: divergence discovers different things, a bandwidth constraint forces those things into compressible, compositional form, and sharing recombines them. This paper tests that intuition in a setting where every term can be measured exactly — evolutionary program synthesis on ARC-AGI-1 — and finds it does not hold at matched compute.

The design, `clebran`, evolves populations ("lineages") of typed programs over Michael Hodel's ARC DSL under pure outcome reward. Multiple lineages are seeded differently and, in the treatment arms, exchange successful material over a blackboard channel. We test three sub-hypotheses: **H1** lineages diverge into different abstractions; **H2** sharing beats not sharing; **H3** a *constrained* channel beats an unconstrained one. Crucially we include a control the intuition usually omits: a *single* agent that grows a central abstraction library from its own solutions using the same compression miner the channel arms use — the library mechanism without the divergence or the channel.

Contributions.

1. A falsified central hypothesis. On a 30-task frontier at 4M candidate-evaluations per run, matched to 0.10% spread, a single agent with a central library is the best arm; the only significant comparison in the study is that it beats the recovered-primitive channel by +0.85 (~2.2 se). Every multi-agent arm lies between the isolated floor and the central-library ceiling.
2. A mechanism: the *divergence tax*. Splitting a fixed compute budget across lineages costs more than communication pays back, and — measurably — degrades the quality of the mined vocabulary, because a lineage that sees only the tasks it solved mines task-idiomatic abstractions.
3. An H3 result: the constraint is null across all four payload types, because adaptive operator selection self-regulates a failing channel regardless of bandwidth — a clean typed-symbolic counter-example to the naive information-bottleneck story.
4. A reusable matched-compute measurement rig: a deterministic typed IR, an audited candidate-evaluation budget, per-task RNG substreams that make single-task probes predict full-run behaviour, and a track record of reporting nulls.

The companion paper (*Not What You Know, How You Learned It*) studies the teaching/curriculum payload and the reorientation the null forces; this paper is the falsification.

2. Substrate and method

The IR and interpreter. A program is an SSA-style sequence of typed statements over Hodel's ~160 grid primitives, mirroring the shape of his hand-written solvers so those solvers parse losslessly into the IR (399/400 reproduce exactly through our deterministic, exception-guarded interpreter). Fitness is pure outcome reward — fraction of demonstration pairs solved

exactly, tie-broken by a foreground/background-balanced per-cell accuracy scaled by an MDL length penalty. No supervised program targets; no neural component.

Lineages and the channel. A lineage is a population evolving under typed mutation/crossover with elitism, a hall of fame, progress-weighted task scheduling, and adaptive operator selection (AOS) that weights each genetic operator by its recent success. Divergence is induced by distinct seeds and primitive-pool subsets. The channel is a blackboard with three regimes — **CH-NONE**, **CH-CONSTRAINED(K,M)** (fragments ≤ K statements, ≤ M messages per epoch), **CH-UNCONSTRAINED** — and four payloads: raw *fragments* (subtrees over given primitives), a *signal* (primitive-reweighting vector), *recovered abstractions* (subroutines a lineage mined from its own solutions and shares as named callable units), and *curricula* (synthetic tasks; studied in the companion paper). The library miner is a Stitch-style compression pass that antiunifies over constants and ranks candidate subroutines by compression utility.

The arms.

arm	description
B0	single population, all compute
B1	N isolated lineages, no channel (union)
B2 / B3	constrained / unconstrained fragment channel
B4	constrained signal channel
B5	**single population + central abstraction library**
B6 / B7	constrained / unconstrained recovered-primitive channel
B8 / B9	constrained / unconstrained curriculum channel

Matched compute. Every arm is capped at the same total candidate-evaluations, metered inside the evaluator; wall-clock never enters a comparison. Reported spreads are ≤ 0.27% across all multi-arm runs, 0.10% for the headline 20-seed frontier run. The primary endpoint is *evolution-phase solves*: the deterministic phase-1 seeding closes the shallow depth-≤2 tasks identically in every arm, so only solves produced by the evolutionary loop can carry a channel effect. All comparisons are reported mean ± standard error over ≥ 5 seeds, and the headline pair over 20.

3. Results

3.1 The Stage-1 first test

The first multi-arm sweep (Stage-1 subset, 5 arms × 5 seeds × 1M evals, 0.27% spread) establishes the pattern. **H1 is supported**: pairwise Jaccard distance over primitive usage is 0.57 ± 0.10, with tasks solved by exactly one lineage in 4/5 seeds — lineages genuinely diverge. **H2 is null**: the fragment channel is statistically indistinguishable from isolation (B2 = B1 = 10.4/30, B0 = 10.6). Channel reuse tops out at 0.17 because the shallow Stage-1 frontier is closed by phase-1 seeding, so the channel has nothing to move. **H3 is directional but not significant** on Stage-1. The null is *structural*, not a budget artifact, which motivated rebuilding the task subset to be deep enough that evolution — and thus the channel — actually engages.

3.2 Bandwidth dose-response

A K×M bandwidth ladder (`none` → K2M1 → K4M3 → K8M10 → `uncon`) on Stage-1 tests the constraint directly. The result is flat and mechanistically informative: evolution-phase solve rate is constant across the ladder (1.67–2.33, all overlapping), and the fragment operator's *share* of offspring is pinned at 0.03 *regardless of bandwidth* — identical even with no channel at all. Widening the channel changes *which* fragments are used (reuse rises 0 → 0.17) but not *how much* splicing happens. The reason is AOS: it weights the fragment operator by its ~2% success versus ~36% for mutation, flooring it near 3% share no matter how much material arrives. **An adaptive learner is robust to unconstrained sharing because it self-constrains; the constraint stops mattering.** This is a clean counter-example, in a typed-symbolic deterministic setting, to the naive information-bottleneck claim that a bandwidth limit is required to prevent a flood of noise.

3.3 The frontier ranking and the central-library control

The headline run is on the frontier subset (25 D2 + 5 D3, all depth ≥ 3), 20 seeds × 4M evals, matched to 0.10%.

arm	evolution-phase solves /30
B5 — single pop + central library	**2.65 ± 1.31**
B0 — single population	2.25 ± 1.07
B7 — recovered-primitive, unconstrained	1.85 ± 1.04
B6 — recovered-primitive, constrained	1.80 ± 1.06
B9 — curriculum, unconstrained	1.65 ± 0.75
B2 — fragment channel	1.60 ± 0.99
B8 — curriculum, constrained	1.60 ± 0.75
B1 — isolated	1.45 ± 0.94

comparison	Δ (se)	verdict
B5 – B6	**+0.85 (0.38)**	**significant (~2.2 se)**
B5 – B0	+0.40 (0.38)	directional, not significant
B6 – B1	+0.35 (0.32)	recovered-primitive transfer, ns
B6 – B7	−0.05 (0.33)	H3 null
B8 – B1	+0.15 (0.27)	curriculum transfer, ns

Three forces explain the ranking. **Splitting compute is expensive** — three isolated lineages (1.45) fall well short of one population (2.25); the small populations converge prematurely. **Communication pays back only part of that tax** — the recovered-primitive channels (≈ 1.8) beat isolation but never reach B0. **The same library mechanism run centrally avoids the tax and adds to single-population search** — B5 lands on top. The recovered-primitive channel's mechanism is real (lineages use, on average, > 1 received abstraction per solve; one lineage discovered a 4-statement mirror-tiling subroutine and used it as a single named call), but the effect is small and underpowered: doubling seeds (8 → 20) shrank rather than confirmed B6 – B2 (+0.50 → +0.20), so it is real-but-small, not a near-miss on significance.

Cost-axis re-score (roadmap §9a). Re-scoring the same 20-seed 4M sweep on **acquisition efficiency** (evals-to-solve) confirms and amplifies the count ranking — solve count is the endpoint Chollet argues is uninformative (*On the Measure of Intelligence*); cost is what can't be bought with priors.

arm	solved/30	evals/solve	× B5 cost
B5	12.45 ± 1.15	**323,848 ± 28,776**	1.0×
B5+planner (paper 3)	12.32 ± 1.00	326,665 ± 24,298	1.01×
B0	2.25 ± 1.07	2,333,510 ± 1,284,369	7.2×
B7	1.90 ± 1.07	2,579,205 ± 1,275,957	8.0×
B6	1.85 ± 1.09	2,631,779 ± 1,241,940	8.1×
B9	1.70 ± 0.86	2,649,320 ± 1,091,437	8.2×
B8	1.65 ± 0.81	2,737,451 ± 1,130,719	8.5×
B2	1.65 ± 1.14	2,748,344 ± 1,194,381	8.5×
B1	1.50 ± 0.95	**3,140,693 ± 1,182,998**	9.7×

B5 and B5+planner are **7–10× more cost-efficient** than every multi-agent arm, on top of solving 6× more tasks. Per-cell productivity is roughly 40× (6× × 7×). **B1 (isolation) is the most expensive failure mode** (3.14M evals/solve — 9.7× B5): the multi-agent tax compounds dramatically when none of the agents can solve anything. **The ordering is robust to the endpoint.** Switching the metric from count to cost preserves the ranking and widens the gap. The only directionally supportive H2 reading on the cost axis is B2 < B1 by ~12% (welch p=0.316); B2 > B0 stays null, with B2 18% *more* expensive than B0 on cost.

The project's central hypothesis — that divergent lineages sharing over a constrained channel beat the alternatives at matched compute — is not supported; the strongest control wins.

3.4 Why divergence hurts vocabulary quality

The vocabulary metrics explain the win from a different angle. B5's mined library is more compositional (topographic similarity 0.33 vs 0.14–0.15 for the multi-agent arms) and more cross-task reused (0.15 vs 0.05). A single agent sees *every* task, so its mined abstractions generalise across them; a lineage that sees only the subset it happened to solve mines task-idiomatic abstractions. Divergence — the premise of H1 — is precisely what *degrades* the shared vocabulary. The signal that lineages use received primitives is genuine; the signal that this improves solve count is not.

3.5 The constraint is null across all four payloads

H3 fails not only for fragments (the dose-response) but for recovered primitives (B6 ≈ B7, Δ = −0.05) and for curricula (B8 ≈ B9, Δ = −0.05). The same AOS self-regulation explains all three: unconstrained sharing cannot crowd out productive mutation because the learner down-weights a failing channel. In this substrate the constraint neither helps nor hurts.

3.6 The discovery bottleneck — four concrete nulls at 1M (2026-09-24)

The transport side of clebran is solved: B5 sits at the top of the frontier ranking (§3.3), is 7–10× more cost-efficient than every multi-agent arm, and the dreaming arm B5D (§3.6.1) builds a library 2× larger, 13× more used, and 124× more cross-task-reused than B5 at 4M, while the frontier remains tied. The discovery side is the open problem: every arm in the rig runs the same DSL, the same interpreter, the same pure-outcome fitness, on the same 30 frontier tasks.

We attacked the discovery bottleneck four times, each through a different *kind* of mechanism — the practice pool, the fitness signal, the task pool, and the **objective** — at 1M evals × 5 seeds each. All four returned null.

3.6.1 #1 Failure-mode dreaming — null at 1M

The dreaming arm B5D produces dreams that exercise the *failure modes* the lineage is currently failing on. Hypothesis: practice on the failure-mode frontier should expand the lineage's reach. Two 1M pilots (5 seeds):

metric (1M, 5 seeds)	B5-1M	B5D-NM-cons	B5D-NM-aggr
frontier solves /30	10.40 ± 1.14	9.80 ± 0.45	**8.80 ± 0.84**
evo-solved /30	3.20 ± 0.84	2.60 ± 0.55	1.60 ± 0.55
evals/solve	97,077	102,230	115,183
welch-t p (vs B5, evals/solve)	—	0.321	**0.007**

The dreaming *is* the same loop as B5D (97% utilization, library 1.7× larger, 21× more used, 5 cross-task reuse events vs B5's 0) but the front-line dream content does not transfer to the frontier: the conservative failure-mode variant regresses to 9.80 vs B5's 10.40 (−0.6, ns), and the aggressive variant regresses to 8.80 vs B5's 10.40 (−1.6, **p = 0.007**). The more aggressively the dreaming is steered at failure modes, the more it costs.

3.6.2 #2 Heterogeneous fitness (B5-strongmdl) — null at 1M

A B5 variant with a stronger MDL pressure (λ = 1.5 vs B5's 1.05), on the hypothesis that parsimony pressure should favor more *compositional* abstractions.

metric (1M, 5 seeds)	B5-1M	B5-strongmdl
frontier solves /30	10.40 ± 1.14	**9.60 ± 1.14**
evo-solved /30	3.20 ± 0.84	**2.40 ± 1.14**
evals/solve	97,077	105,468

B5-strongmdl *regresses* (−0.80 frontier, −0.80 evo, both ~1 SE), is *more* expensive (105K vs 97K evals/solve), and the *fewer* evo-phase solves means the MDL pressure pushed the lineage to shorter *phase-1-seeded* programs, not new compositions. The default B5 is well-tuned; stronger MDL on the same primitives doesn't open new regions of program space.

3.6.3 #3 Task-subset specialization — null at 1M

Two arms × 5 seeds × 1M each. The *hard* arm runs B5 on the 15 stage1 tasks B5-4M (15 seeds) never solved. The *random* arm runs B5 on 15 random stage1 tasks. Per-task compute: 1M/15 = **67k/task** in both arms, twice the B5-1M full-stage1 baseline's 33k/task.

metric (1M, 5 seeds)	hard15 arm	random15 arm	B5-1M full stage1	B5-4M full stage1
test-exact / 15-task subset	**0.00 ± 0.00**	**5.60 ± 0.55**	0.00 / 4.60	0.00 / 3.53
frontier / 15-task subset	0.20 ± 0.45	5.80 ± 0.84	0.20 / 4.60	0.20 / 5.27
train-only overfits	1 (1f85a75f)	1 (0b148d64)	1 (1f85a75f)	3 (1f85a75f ×3)
welch-t p (vs B5-1M on subset)	1.000	**0.007**	—	—

Two findings in one:

- *Specialization on hard tasks fails.* hard15 gets 0 test-exact discoveries; B5-4M (15 seeds, 133k/task) gets 0 test-exact on the same subset. The hard15 set is *genuinely beyond B5's reach at these budgets* — the only frontier "solves" are train-only overfits on `1f85a75f`, which the test pairs catch. No specialization trick can rescue a search that's out of room.
- *Specialization on random tasks = per-task compute.* random15 gets 5.6 test-exact vs B5-1M's 4.6 on the same subset (p = 0.007). But the specialized arm has 2× per-task compute (67k vs 33k); a 2× compute-on-easy-tasks increase is the natural explanation for ~1 more solve on a frontier of easy D1 tasks. No signal for "specialization helps on a complementary subset" beyond the compute redistribution.

3.6.4 #4 Novelty search (B5N) — null at 1M

The fourth attack varies the *objective* rather than the practice pool, fitness signal, or task pool: a B5 lineage that scores on **behavioral novelty** (distinct-output novelty: 1.0 for new output hashes, decaying toward 0 as the same output is produced by more programs) instead of task accuracy. Hypothesis: novelty search traverses a different region of program space than accuracy-driven search; even if it doesn't beat B5 on solve count, it might find *unique-to-it* test-exact solves that B5 misses.

Result: 5 seeds × 1M evals. The novelty arm finds exactly the 6 easy tasks B5-1M also finds (100% overlap), with 0 unique-to-novelty tasks. All 5 seeds *degenerated* partway through — novelty scoring as the *primary* objective doesn't reward accuracy-improving mutations, so once the per-task novelty archive filled with distinct outputs, subsequent programs produced duplicates and novelty plateaued near 0. The lineage made no accuracy-driven progress in evolution; all 6 B5N-solves came from the seeding phase. Read honestly, this is *also* a null result — but with a strong mechanism-level reading: **novelty search as a standalone objective can't drive B5's evolution** because it doesn't reward improvement.

metric (1M, 5 seeds, partial)	B5N novelty	B5-1M accuracy
test-exact solves / seed (mean)	2.00	10.20
tasks ever solved across all seeds	6	24
tasks unique to this arm	**0**	6
overlap with B5-1M	6 / 6 = 100%	n/a
seeds that degenerated	5 / 5	n/a

3.6.5 Reading the four together

None of the four mechanisms expanded B5's frontier at 1M:

#	mechanism	kind of attack	result	what we learned
1	failure-mode dreaming	change the *practice pool* (targeted)	null	more aggressive targeting → significant regression (p=0.007)
2	heterogeneous fitness	change the *fitness signal* (parsimony pressure)	null	stronger MDL → mild regression; produces *shorter*, not *more compositional*, programs
3	task-subset specialization	change *what the lineage searches on* (subset)	null	hard set out of reach; random subset gain = compute
4	novelty search (B5N)	change the *objective* (behavioral diversity)	null	finds the same 6 easy tasks as B5-1M (100% overlap); novelty-as-objective degenerates

The four attack the same search space from four different angles: practice pool, fitness signal, task pool, **objective**. All four return null at 1M. The cheapest reading is that **no mechanism operating *inside* the current DSL/interpreter/fitness on the current 30-task frontier will expand B5's reach at 1M, regardless of which axis you vary.** The remaining bets — learned L1 as proposer (#5, programmatic L1), operator self-modification (#6, learning to learn), L3 evoker (#7, 9f.4 world-modeling capstone) — all attack the problem by

adding a *new dimension* the lineage can search (a new primitive distribution, a new operator pool, a new world model), rather than redistributing budget across the same DSL. **The structural-change bet is now the dominant remaining strategy on this rig.** If #5–#7 also return null, the menu closes and the project's natural next move is to bring in a genuinely new substrate (ARCTraj, ARC-AGI-3) — not to keep attacking the same DSL with new mechanisms.

Artifacts: the four pilots are frozen at `experiments/2026-08-31_b5d_nm_1m/`, `experiments/2026-08-31_b5d_nm_aggressive_1m/` (#1), `experiments/2026-08-31_b5_heterogeneous_1m/` (#2), `experiments/2026-09-22_task_subset_specialization/` (#3), and `experiments/2026-09-24_novelty_search/` (#4). The cost-axis loader (`clebran/metrics/cost.py`) now picks up the #3 arms so the menu pilots roll into the cost-axis ranking; #4's truncated logs (5 seeds that all degenerated before 1M) are at `logs/b5n_1m_seed{1..5}.jsonl`.

4. Discussion

A caveat on the B5 attribution. B5's win over B0 (+0.40) is directional, not significant, and B5 differs from the multi-agent arms in two ways at once: it is centralised *and* it builds its library from a single population with ~2.65 mean solves versus ~0.6–0.7 per lineage — a ~4× larger library pool. The cleanest isolation of the library mechanism is a *B0 + library* arm (single population, same miner, no multi-agent framing); until it is run, the defensible claim is that *centralisation plus the library mechanism* dominates *divergence plus any channel*, not that the library mechanism alone is what wins. The B5 − B6 contrast that *is* significant compares the same library mechanism under centralisation versus divergence-plus-channel, which is exactly the falsification we claim.

What to keep, what to rethink. The negative result has a positive corollary: the *library* design — the compression miner, the MDL pressure, the Stitch-style antiunification — is the component of `clebran` worth keeping, and B5 is a strong "abstraction reuse without divergence" baseline any multi-agent framing must beat, not merely match. The multi-agent framing, the channel, and the constraint regime are the parts to rethink.

Implications for related literatures. For *emergent communication*, this is a clean refutation of the naive information-bottleneck story in a typed symbolic setting: bandwidth-limited channels between divergent agents do not induce better compositionality at matched compute, because an adaptive learner self-constrains. This does not refute the cultural-evolution account of compression driving compositional structure (which involves iterated transmission rather than a single inter-population channel), but it argues the simple story is incomplete even in a non-neural, deterministic setting — echoing prior neural results that successful emergent communication need not entail compositionality. For *library learning*, the result is consistent with a Stitch/DreamCoder-style central learner being a strong baseline; the novel element here

is the matched-compute comparison against divergent multi-agent baselines, and the win is significant.

The measurement rig is the asset. A cheap critique of any null in evolutionary synthesis is "you under-powered the channel." The power check doubled seeds and raised budget; the effect shrank and the standard error narrowed, so the result is real-but-small, not a near-miss. The rig — audited candidate-evaluation budgets, per-task RNG substreams that make single-task debugging predict full-run behaviour, a deterministic interpreter with a 399/400 reproduction gate — is what makes the null trustworthy. A methodological lesson also falls out: report *cost-to-solve* alongside solve count, and always include a *do-nothing* control; both are developed in the companion paper.

5. Limitations

The result is scoped to one substrate (typed evolutionary program synthesis on ARC), one frontier subset, and matched budgets where solve counts are small (1–3 evolution solves per seed). H3 is *partially confounded with AOS*: because AOS regulates channel intake regardless of bandwidth, H3 may be untestable in this substrate rather than merely null. The B5 – B0 library-pool confound (above) is not yet controlled. The eval split of ARC has not been touched, so no claim is made about generalisation to genuinely unseen tasks. These bound the claim to what it is: at matched compute, on this substrate, a central library significantly beats divergent constrained communication, and the divergence tax is the mechanism.

6. Conclusion

We set out to test whether divergent populations exchanging subroutines over a constrained channel beat the alternatives at matched compute, with baselines strong enough that a null would be trustworthy. The experiment delivered a falsification: a single agent that grows a central abstraction library is the best arm, and the only significant comparison in the study is that it beats the recovered-primitive channel by +0.85 (~2.2 se). Every multi-agent arm lies between isolation and the central library; the constraint is null across all four payload types; and divergence, though real, degrades vocabulary quality rather than improving it. Communication is a transport mechanism, not a discovery mechanism, and in this substrate transport does not pay back the cost of dividing the compute.

The follow-on sweep of the discovery-bottleneck menu (§3.6) closes the same way. Four concrete attacks on whether any mechanism *inside* the central learner can expand its frontier — practice pool, fitness signal, task pool, and objective — all return null at 1M × 5 seeds each. The transport side of clebran is solved; the discovery side is open. The remaining bets on the menu all change the same scope, and the natural next substrate for the project is one where the search

space itself is wider (ARCTraj, ARC-AGI-3). The result is what the design's baselines were built to make believable, and it is the result any future multi-agent program-synthesis proposal in this area has to address.